

Build a Tiny Database in Java

<https://medium.com/javarevisited/build-a-tiny-database-in-java-ca6d3f06e115>

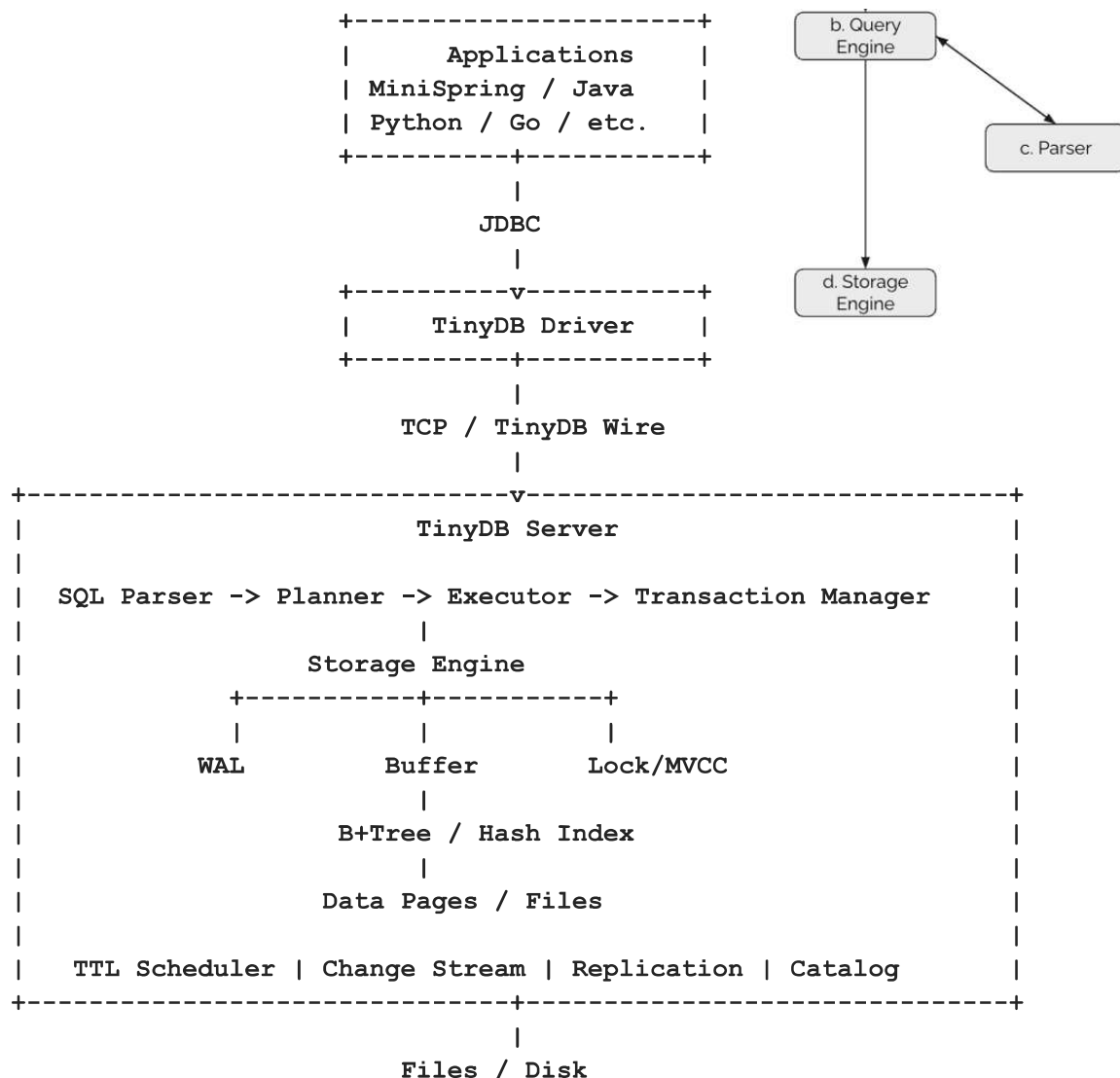
```
Tinysql> SELECT A,B FROM T1 WHERE A=1;
+-----+
| a | b |
+-----+
| 1 | Alice |
+-----+
exit;
```

Components

1. CLI
2. Query Parser
3. Query Engine
4. Storage Engine

It has been a while since I wrote: [“Build a Tiny Compiler in Java.”](#) Now, I am back with a new article on a tiny database written in Java. This code is a bare-minimum implementation designed for learning purposes. The work is inspired by [“SimpleDB”](#) by Edward Sciore.

This code is a bare-minimum implementation designed for learning purposes. Basically, I would like information on the following:



1. Command Line Interface

In the **CLI loop**, we call the query engine **doQuery** or **doUpdate** function based on the SQL query's prefix keyword. Yes, it is pretty naive :).

*The query engine abstracts all the downstream interactions. **Query Engine** internally calls **Parser** to parse the SQL statements and convert it to Domain objects. The **Domain Objects** are then used by the **Query Planner** to create a **query execution plan** based on some rules or heuristics. The Query Plan is executed by the **Execution Engine**. The Execution engine talks to **Storage Engine** by **scanning** disk files and reading the table records. These records are then send back to Query Engine via **Iterator Pattern**. The Query Engine applies **filters** based on **Selection** and **Projection** Operators defined in the **Query Plan** and finally sends it back to the **CLI** for printing the table output.*

```
private static void cliLoop(IQueryEngine qe) {
    Scanner scanner = new Scanner(System.in).useDelimiter(";");
    TablePrinter tablePrinter = new TablePrinter();
    while (true) {
        System.out.print("tinysql> ");
        String sql = scanner.next().replace("\n", " ").replace("\r", "").trim();

        TableDto result;
        if (sql.startsWith("exit")) break;
        else if (sql.startsWith("select")) result = qe.doQuery(sql);
        else result = qe.doUpdate(sql);

        if (result.message.isEmpty()) tablePrinter.print(result);
        else System.out.println(result.message);
    }
    scanner.close();
}
```

2. Parser

This layer will parse SQL, tokenize the inputs, create the domain object, and send it to the next layer, ie Query Engine. Our parser currently supports `SELECT`, `CREATE`, `INSERT`, `DELETE`, `UPDATE` & `DROP`.

We are using the pre-compiled MySQL ANTLR parser from [Apache ShardingSphere](#) (Kudos to the community!).

We could also write a custom parser from scratch to parse MySQL Dialect. But for now, we are mainly focusing on writing a tiny database rather than a functional parser.

The SQL query after being parsed is converted to Domain Objects.

```
// SQL: SELECT AGE,NAME FROM STUDENTS WHERE AGE=18;
// AGE,NAME -> Fields
// STUDENTS -> Table
// AGE=18 -> Predicate
public class QueryData {
    private List<String> fields;
    private String table;
    private A_Predicate pred; // Predicate is a Boolean combination of terms.
}
```

```
// SQL : INSERT INTO STUDENTS (NAME, AGE) VALUE ("Bob", 18)
public class InsertData {
    private String tblname;
    private List<String> flds;
    private List<D_Constant> vals;
}
```

We have used the MySQL Dialect here. Using the `Lexer` and `Parser` we convert the SQL statements to the Domain objects, which will be used by the query engine.

```
public class MySqlParser implements IParser {
    MySqlStatementVisitor sqlStatementVisitor;

    public MySqlParser(String sql) {
        MySQLLexer lexer = new MySQLLexer(CharStreams.fromString(sql));
        MySQLStatementParser parser = new MySQLStatementParser(new CommonTokenStream(lexer));

        sqlStatementVisitor = new MySqlStatementVisitor(parser);
        sqlStatementVisitor.visit(parser.execute());
    }

    // Returns a QueryData Domain Object
    @Override
    public QueryData queryCmd() {
        return (QueryData) sqlStatementVisitor.getValue();
    }

    // Object can be of type DeleteData, ModifyData, InsertData etc.
    @Override
    public Object updateCmd() {
        return sqlStatementVisitor.getValue();
    }
}
```

A simple example of how the ANTLR visitor pattern works is shown below. You can add a custom visitXXX() implementation depending on which fields (from the SQL) you want to extract for creating a domain object.

```
// The ANTLR visitor class for Parsing SQL Statements.
public class MySqlStatementVisitor extends MySQLStatementBaseVisitor {
    private final MySQLStatementParser parser;
    private COMMAND_TYPE commandType;
    private String tableName;

    // Populate COMMAND TYPE
    @Override
    public Object visitCreateIndex(MySQLStatementParser.CreateIndexContext ctx) {
        commandType = COMMAND_TYPE.CREATE_INDEX;
        return super.visitCreateIndex(ctx);
    }

    @Override
    public Object visitCreateTable(MySQLStatementParser.CreateTableContext ctx) {
        commandType = COMMAND_TYPE.CREATE_TABLE;
        return super.visitCreateTable(ctx);
    }
    ....

    // Populate TableName
    @Override
    public Object visitTableName(MySQLStatementParser.TableNameContext ctx) {
        this.tableName = ctx.name().getText();
        return super.visitTableName(ctx);
    }
    ....
}
```

Once you populate the fields inside the domain object, you are good to pass it to the Query Engine.

3. Query Engine

This layer mainly consists of 3 parts:

- **Query Optimizer** (Finds the JOIN cost and creates an optimal query plan)
- **Execution Engine** (Interacts with Storage engine using Read/Write or ReadOnly scans)
- **Catalog/Metadata Manager** (Holds the table schema information)

3.a. Query Optimizer

This layer is responsible for calculating JOIN cost and finding an optimal query-plan to fetch data from the underlying Storage Engine.

The Optimizer (in Oracle), **Query Optimizer** (in SQL Server, MySQL), or **Query Planner** (in PostgreSQL) translates the SQL statement to an execution plan.

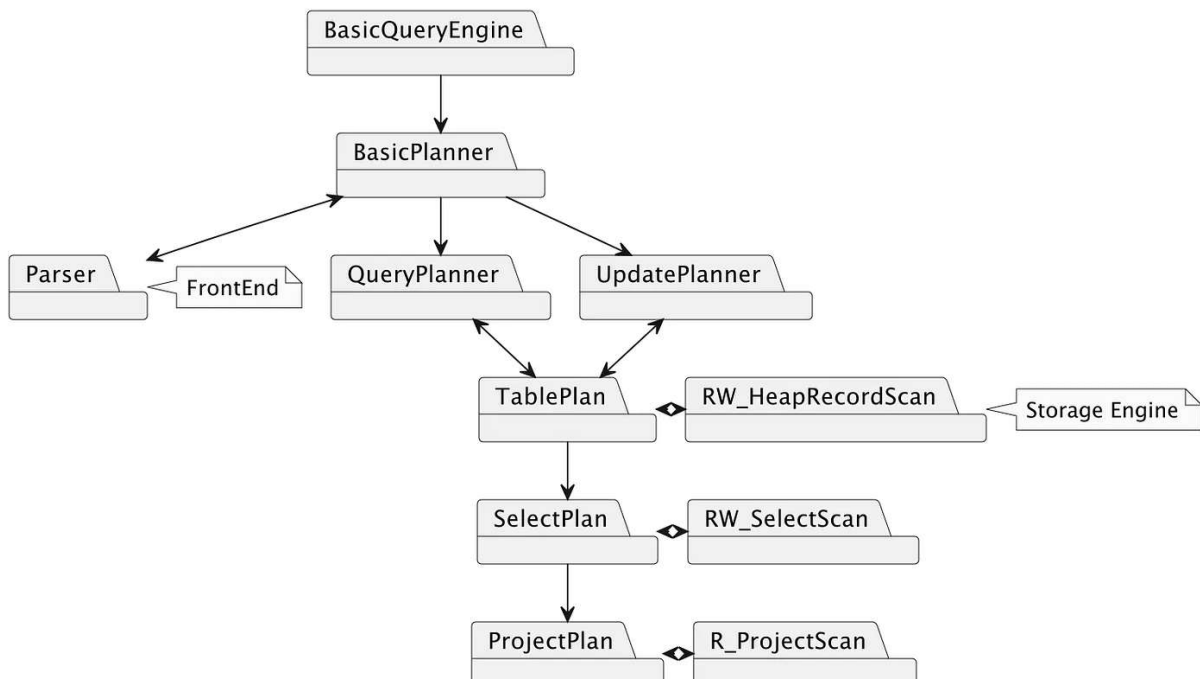
There are generally two kinds of Optimizers:

1. **Rule-Based Optimizer (RBO)**: Rule-Based Optimizers follow a strict set of rules to create an execution plan — e.g., always use an index if possible.
2. **Cost-Based Optimizer (CBO)**: Cost-Based Optimizers generate many different execution plans, apply a cost model to all of them and select the one with the best cost value for execution.

We have used a simple rule-based query optimizer, that will use Index File if the column has been indexed.

Instead of Query Engine, we could also use Apache Calcite to perform both Parsing and Query Optimization. I did try it out and it works pretty well.

Below is a high-level execution flow of the Query Engine, Execution Engine, and Storage Engine.



Basic Query Engine

The Query Engine has 2 main functions, `doQuery`, and `doUpdate` (we invoked them in the CLI code)
The QueryEngine's code simply invokes the BasicPlanner `createQueryPlan` or `executeUpdate` and prints the results.

```
//BasicQueryEngine.java
public TableDto doQuery(String sql) {
    // Call Planner
    Transaction tx = newTx();
    Plan p = planner.createQueryPlan(sql, tx);

    // Print the result
    RORecordScan s = p.open();
    List<String> columnNames = p.schema().fields();
    List<List<String>> rows = new ArrayList<>();
    while (s.next()) {
        List<String> row = new ArrayList<>();
        for (String field : columnNames)
            row.add(s.getVal(field).toString());
        rows.add(row);
    }

    s.close();
    tx.commit();

    return new TableDto(columnNames, rows);
}

public TableDto doUpdate(String sql) {
    // Call Planner
    Transaction tx = newTx();
    int updatedRows = planner.executeUpdate(sql, tx);
    tx.commit();

    // Print the result
    String message = updatedRows + " " +
        (updatedRows == 1 ? "row" : "rows") +
        " updated.";
    return new TableDto(message);
}
```

Basic Planner (Facade Pattern)

BasicPlanner is an abstraction for 2 types of planners, the Query Planner and Update Planner.
Depending on the parser's domain object type, we determine which function to invoke.

```
//BasicPlanner.java
public BasicPlanner(QueryPlanner queryPlanner, UpdatePlanner updatePlanner) {
    this.queryPlanner = queryPlanner;
    this.updatePlanner = updatePlanner;
}

public Plan createQueryPlan(String qry, Transaction tx) {
    IParser parser = new MySqlParser(qry);
    QueryData data = parser.queryCmd();
    return queryPlanner.createPlan(data, tx);
}

public int executeUpdate(String cmd, Transaction tx) {
    IParser parser = new MySqlParser(cmd);
    Object data = parser.updateCmd();
    if (data instanceof InsertData) return updatePlanner.executeInsert((InsertData) data, tx);
    else if (data instanceof DeleteData) return updatePlanner.executeDelete((DeleteData) data, tx);
    else if (data instanceof ModifyData) return updatePlanner.executeModify((ModifyData) data, tx);
    else if (data instanceof CreateTableData) return updatePlanner.executeCreateTable((CreateTableData)
data, tx);
    else if (data instanceof CreateIndexData) return updatePlanner.executeCreateIndex((CreateIndexData)
data, tx);
    else return 0;
}
```

We have 2 implementations of Planner here

- **Basic** (Naive: No optimization)
- **Rule Based** (Rule: If the query contains an indexed column, use that index to speed up the Query)

Basic Query Planner

We will now cover the **Basic** Planner. **Rule-based** Planner contains minor changes to accommodate the index rule.

The Basic Query Planner mainly uses the 3 items below.

- **TablePlan**: Read Data from the Storage Engine using Iterator Pattern
- **SelectPlan**: Filter out the rows based on the predicate condition
- **ProjectPlan**: Filter out the columns based on fields in the query.

```
public class BasicQueryPlanner implements QueryPlanner {
    private MetadataMgr mdm;

    public BasicQueryPlanner(MetadataMgr mdm) {
        this.mdm = mdm;
    }

    public Plan createPlan(QueryData data, Transaction tx) {
        //Step 1: Create the plan
        Plan p = new A_TablePlan(tx, data.table(), mdm);

        //Step 2: Add a selection plan for the predicate
        p = new B_SelectPlan(p, data.pred());

        //Step 3: Project on the field names
        p = new C_ProjectPlan(p, data.fields());
        return p;
    }
}
```

Basic Update Planner

The Basic Query Planner mainly uses the 3 items below.

- **TablePlan**: (RWRecordScan)plan.Open() enables us to read/write data from/to the storage engine.
- **MetadataMgr**: This is responsible for providing the catalog information (table schema) of a table.

```
//BasicUpdatePlanner.java
private MetadataMgr mdm;

public BasicUpdatePlanner(MetadataMgr mdm) { this.mdm = mdm; }

public int executeCreateTable(CreateTableData data, Transaction tx) {
    mdm.createTable(data.tableName(), data.newSchema(), tx);
    return 0;
}

public int executeCreateIndex(CreateIndexData data, Transaction tx) {
    mdm.createIndex(data.indexName(), data.tableName(), data.fieldName(), tx);
    return 0;
}

public int executeInsert(InsertData data, Transaction tx) {
    // Step 1: Create a table plan
    Plan p = new A_TablePlan(tx, data.tableName(), mdm);

    // Step 2: Open the plan in RW mode and seek to the next insert position
    RWRecordScan scan = (RWRecordScan) p.open();
    scan.seekToInsertStart();

    // Step 3: read values from the data (domain object)
    // Step 4: Write these values to the RW scan curr position
    Iterator<D_Constant> iter = data.vals().iterator();
    for (String fldname : data.fields()) {
        D_Constant val = iter.next();
        scan.setVal(fldname, val);
    }
    scan.close();
    return 1;
}
```

Plans

Plan class is used to encapsulate relational algebra and the cost (like the number of blocks accessed etc) of operation. The query plan from Query Optimizer, is used to get an efficient physical plan (ie scans) to read data efficiently from the disk.

Since we are not using a cost-based planner, we won't be making the best use of statistics provided by Plans (eg blocksAccessed).

```
public interface Plan {
    RORecordScan open();
    TableDefinition schema();
    int blocksAccessed();
}
```

```
// TablePlan.java
public TablePlan(Transaction tx, String tblname, MetadataMgr md) {
    this.tblname = tblname;
    this.tx = tx;
    recordValueLayout = md.getLayout(tblname, tx);
}

public RORecordScan open() {
    // NOTE: The Place where Query Engine interacts with StorageEngine
    return new HeapRWRecordScan(tx, tblname, recordValueLayout);
}
```

```
// SelectPlan.java
public SelectPlan(Plan p, Predicate pred) {
    this.p = p;
    this.pred = pred;
}

public RORecordScan open() {
    RORecordScan s = p.open();
    return new Select_RWRecordScan(s, pred);
}
```

3.b. Execution Engine

The execution engine accesses the required data from the underlying storage system, whether it's a disk-based file system or an in-memory database. This involves retrieving data from tables, indexes, or other data structures based on the query predicates and JOIN conditions.

For this project, we only have Select and Project Scans inside Execution Engine.

Scans

Upon `plan.open()` , we get a `scan` object.

```
// Select_RWRecordScan.java
public Select_RWRecordScan(RORecordScan s, Predicate pred) {
    this.s = s;
    this.pred = pred;
}

public boolean next() {
    // NOTE: Row filtering logic
    while (s.next()) {
        if (pred.isSatisfied(s)) {
            return true;
        }
    }
    return false;
}
```

```
//Project_RORecordScan.java
public Project_RORecordScan(RORecordScan s, List<String> fieldlist) {
    this.s = s;
    this.fieldlist = fieldlist;
}

public boolean next() {
    return s.next();
}

public int getInt(String fldname) {
    if (hasField(fldname))
        return s.getInt(fldname);
    else
        throw new RuntimeException("field " + fldname + " not found.");
}

public String getString(String fldname) {
    if (hasField(fldname))
        return s.getString(fldname);
    else
        throw new RuntimeException("field " + fldname + " not found.");
}

// NOTE: Column filtering logic
// Called in BasicQueryEngine --> doQuery()
public D_Constant getVal(String fldname) {
    if (hasField(fldname))
        return s.getVal(fldname);
    else
        throw new RuntimeException("field " + fldname + " not found.");
}
```

3.c. Catalog Manager

This mainly contains

- **Index Manager** (responsible for giving index information)
- **Stats Manager** (responsible for maintaining stats like the number of blocks accessed etc, which can be used for heuristic cost analysis)
- **Table Manager** (responsible for providing table schema information)

For this project, we only have Select and Project Scans inside Execution Engine.

Table Definition (Logical Layout)

The Table definition contains information like

field names, field types, and the length of the fields (in the case of VARCHAR)

```
public class TableDefinition {  
  
    private final List<String> fields = new ArrayList<>();  
    private final Map<String, FieldInfo> info = new HashMap<>();  
  
    public void addField(String fldname, int type, int length) {  
        fields.add(fldname);  
        info.put(fldname, new FieldInfo(type, length));  
    }  
  
    public void addIntField(String fldname) {  
        addField(fldname, INTEGER, 0);  
    }  
  
    public void addStringField(String fldname, int length) {  
        addField(fldname, VARCHAR, length);  
    }  
}
```

Table Physical Layout (Physical Layout)

Structure to hold the Physical layout (Byte Offset) of the records. It also contains the slot size, which is the number of bytes required for a row. We are using fixed-size slot sizes for simplicity, ie even VARCHAR will have a fixed length.

```
public class TablePhysicalLayout {  
    private TableDefinition tableDefinition;  
    private Map<String, Integer> offsets;  
    private int slotsize;  
}
```

Table Manager

It is responsible for storing information about the tables.

Let's take an example SQL here, which creates a table T1.

```
create table T1 ( A int, B varchar(9) );
> 0 rows updated.

insert into T1 (A, B) values (1, 'Alice');
> 1 row updated.

insert into T1 (A, B) values (2, 'Bob');
> 1 row updated.

select A,B from T1;
>
+---+-----+
| a | b      |
+---+-----+
| 1 | Alice  |
| 2 | Bob    |
+---+-----+
```

The database creates 2 internal tables for holding metadata about the user tables. It is similar to the `information_schema`.

tinydb_tables:

```
tinysql > select tblname, slotsize
from tinydb_tables;
```

```
+-----+-----+
| tblname | slotsize |
+-----+-----+
| tinydb_tables | Alice |
| tinydb_columns | Bob |
| idxcat | |
| T1 | |
+-----+-----+
```

tinysql>

tinydb_columns:

```
tinysql > select tblname, fldname, type, length, offset
from tinydb_columns;
```

```
+-----+-----+-----+-----+-----+
| tblname | fldname | type | length | offset |
+-----+-----+-----+-----+-----+
| tinydb_tables | tblname | 12 | 16 | 4 |
| tinydb_tables | slotsize | 4 | 0 | 24 |
| tinydb_columns | tblname | 12 | 16 | 4 |
| tinydb_columns | fldname | 12 | 16 | 24 |
| tinydb_columns | type | 4 | 0 | 44 |
| tinydb_columns | length | 4 | 0 | 48 |
| tinydb_columns | offset | 4 | 0 | 52 |
| idxcat | indexname | 12 | 16 | 4 |
| idxcat | tablename | 12 | 16 | 24 |
| idxcat | fieldname | 12 | 16 | 44 |
| T1 | A | 4 | 0 | 4 |
| T1 | B | 12 | 9 | 8 |
+-----+-----+-----+-----+-----+
```

tinysql>

Files: In our design, we create a new file per table and index. So `T1`, `tinydb_columns`, and `tinydb_tables` have their own file.

`idxcat` is for index information.

```
tinydb/
├── idxcat.tbl
├── T1.tbl
├── tinydb_columns.tbl
└── tinydb_tables.tbl
```

```

public class TableMgr {
    // The max characters a tablename or fieldname can have.
    public static final int MAX_NAME = 16;
    private TablePhysicalLayout tcatRecordValueLayout, fcatRecordValueLayout;

    // Register a new table having the specified name and schema.
    // It updates tinydb_tables.tbl and tinydb_columns.tbl
    public void createTable(String tblname, TableDefinition sch, Transaction tx) {
        TablePhysicalLayout recordValueLayout = new TablePhysicalLayout(sch);

        // write to tinydb_tables.tbl file
        HeapRWRecordScan tcat = new HeapRWRecordScan(tx, "tinydb_tables", tcatRecordValueLayout);
        tcat.seekToInsertStart();
        tcat.setString("tblname", tblname);
        tcat.setInt("slotsize", recordValueLayout.slotSize());
        tcat.close();

        // write to tinydb_columns.tbl file
        HeapRWRecordScan fcat = new HeapRWRecordScan(tx, "tinydb_columns", fcatRecordValueLayout);
        for (String fldname : sch.fields()) {
            fcat.seekToInsertStart();
            fcat.setString("tblname", tblname);
            fcat.setString("fldname", fldname);
            fcat.setInt("type", sch.type(fldname));
            fcat.setInt("length", sch.length(fldname));
            fcat.setInt("offset", recordValueLayout.offset(fldname));
        }
        fcat.close();
    }

    // Retrieve the layout of the specified table from the catalog
    public TablePhysicalLayout getLayout(String tblname, Transaction tx) {
        // **Step 1**: Read from tinydb_tables.tbl
        int size = -1;
        HeapRWRecordScan tcat = new HeapRWRecordScan(tx, "tinydb_tables", tcatRecordValueLayout);
        // [tableName1, slotsize1] | [tableName2, slotsize2] | [tableName3, slotsize3]

        while (tcat.next()) {
            if (tcat.getString("tblname").equals(tblname)) {
                size = tcat.getInt("slotsize");
                break;
            }
        }
        tcat.close();

        // **Step 2**: Read from tinydb_columns.tbl
        TableDefinition sch = new TableDefinition();
        Map<String, Integer> offsets = new HashMap<String, Integer>();
        HeapRWRecordScan fcat = new HeapRWRecordScan(tx, "tinydb_columns", fcatRecordValueLayout);
        // [tableName1, A, int, 4, offset] | [tableName1, B, varchar, 9, offset] | [tableName3, fldname, type, length
offset]
        while (fcat.next()) {
            if (fcat.getString("tblname").equals(tblname)) {
                String fldname = fcat.getString("fldname");
                int fldtype = fcat.getInt("type");
                int fldlen = fcat.getInt("length");
                int offset = fcat.getInt("offset");
                offsets.put(fldname, offset);
                sch.addField(fldname, fldtype, fldlen);
            }
        }
        fcat.close();

        // **Step 3**: Using schema and offset, populate TablePhysicalLayout
        return new TablePhysicalLayout(sch, offsets, size);
    }
}

```

NOTE: The getLayout() is used inside TablePlan to create RecordScan, which is then used to read data from the storage engine

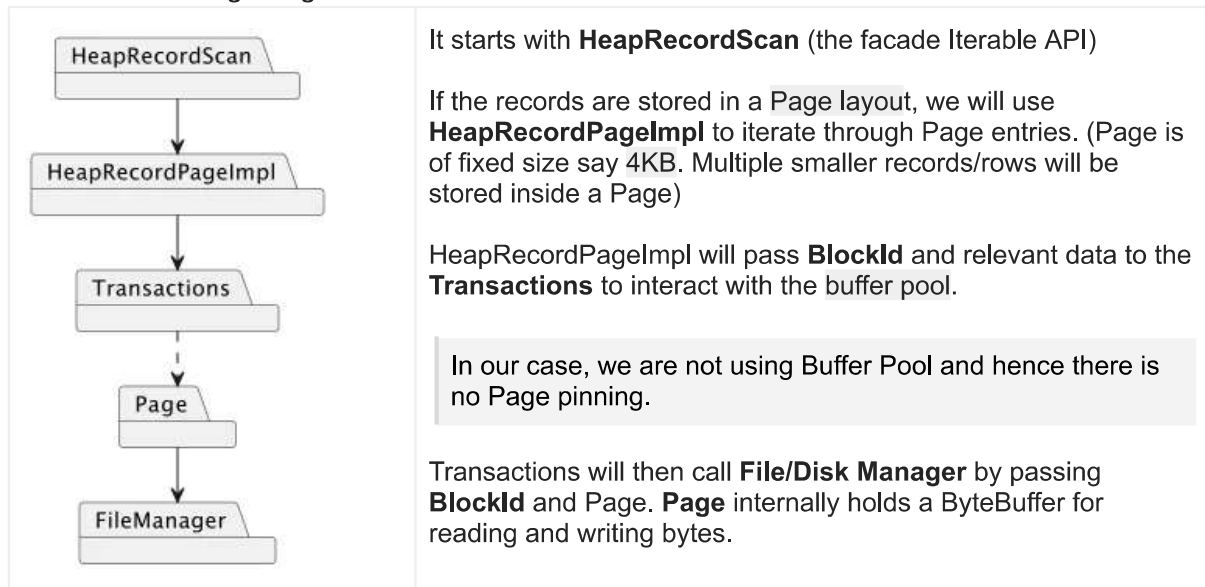
Storage Engine

This is a key-value store module that can talk to the underlying file system and perform CRUD (Create Read Update Delete) operations on database disk files. We are using a Heap Page Organization (ie Data is stored as Blocks without sorting), instead of the traditional B+Tree Page Organization. Since we create a B+Tree index on top of the Heap based data

Different DBMS manage pages on disk files in different ways.

- Heap File Organization (Our Approach)
- Tree File Organization (Sorted order tree)
- Sorted File Organization (SST)
- Hashing File Organization

The overall storage engine flow is as below



Components

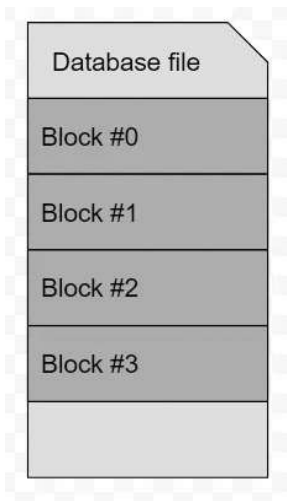
The storage engine mainly contains

- the data module
- the index module

Data Module

The data module mainly covers the data persistence layer of the database.

0. Block Id



The data is stored inside a database file in the form of fixed-sized Blocks. It is read into the memory in the form of Page. BlockId represents a unique identifier to read a file block.

```
public class BlockId {
    private final String fileName;
    private final int blockNumber;
}
```

1. HeapRecordScan

We have interfaces for RO and RW Record scans.

```
// The interface implemented by all query scan.
public interface RORecordScan {
    // Position the scan before the first record. Subsequent
    // call to next() will return the first record.
    public void seekToQueryStart();

    // Move the scan to the next record.
    public boolean next();

    // Use getters to read the feild values
    public int getInt(String fldname);
    public String getString(String fldname);
    public D_Constant getVal(String fldname);
    public boolean hasField(String fldname);

    public void close();
}
```

```
// The interface implemented by all updatable scans.
public interface RWRecordScan extends RORecordScan {
    // Seek to the next insert position in the file.
    public void seekToInsertStart();

    // Return the id of the current record.
    public RecordKey getRid();

    // Position the scan so that the current record having the specified id.
    public void seekTo(RecordKey recordKey);

    // Use setters to set the values
    public void setVal(String fldname, D_Constant val);
    public void setInt(String fldname, int val);
    public void setString(String fldname, String val);

    public void delete();
}
```

The concrete implementation of **RWRecordScan** is shown below in **HeapRWRecordScan**.

This class mainly deals with traversing the file records, seeking to the right position and finally reading or writing records at that position.

```

// Provides the enumeration (iterator) for records stored in the Disk.
public class HeapRWRRecordScan implements RWRRecordScan {
    private final Transaction tx;
    private final TablePhysicalLayout recordValueLayout;
    private HeapRecordPageImpl rp;
    private final String filename;
    private int currentSlot;

    public HeapRWRRecordScan(Transaction tx, String tblname, TablePhysicalLayout recordValueLayout) {
        this.tx = tx;
        this.recordValueLayout = recordValueLayout;
        filename = tblname + ".tbl";

        if (tx.blockCount(filename) == 0) {
            createAndMoveToNewBlock();
        } else {
            moveToBlock(0);
        }
    }

    // Move to 0th Block
    public void seekToQueryStart() {
        moveToBlock(0);
    }

    // currentSlot maintains last read/write position.
    public void seekToInsertStart() {
        currentSlot = rp.insertAfter(currentSlot);
        while (currentSlot < 0) {
            if (atLastBlock()) {
                createAndMoveToNewBlock();
            } else {
                moveToBlock(rp.getBlockId().getBlockNumber() + 1);
            }
            currentSlot = rp.insertAfter(currentSlot);
        }
    }

    // Move to a new slot within the Block
    // If it is the last block, return false.
    public boolean next() {
        this.currentSlot = rp.findSlotAfter(this.currentSlot);
        while (currentSlot < 0) {
            if (atLastBlock()) {
                return false;
            }
            moveToBlock(rp.getBlockId().getBlockNumber() + 1);
            this.currentSlot = rp.findSlotAfter(this.currentSlot);
        }
        return true;
    }

    public void delete() {
        rp.delete(currentSlot);
    }

    ...

    public void setInt(String fldname, int val) {
        rp.setInt(currentSlot, fldname, val);
    }

    ...

    public int getInt(String fldname) {
        return rp.getInt(currentSlot, fldname);
    }
}

```

2. HeapRecordPageImpl

This class is used inside HeapRWRecordScan. This class is used to read a single block (using BlockId) and read the records within that block. This class passes BlockId, Field Position, and Payload to the Transaction class from reading and writing records.

```
public class HeapRecordPageImpl {
    public static final int EMPTY = 0, USED = 1;
    private Transaction tx;
    private BlockId blockId;
    private TablePhysicalLayout recordValueLayout;

    public HeapRecordPageImpl(Transaction tx, BlockId blockId, TablePhysicalLayout
recordValueLayout) {
        this.tx = tx;
        this.blockId = blockId;
        this.recordValueLayout = recordValueLayout;
    }

    public int findSlotAfter(int slot) {
        return searchAfter(slot, USED);
    }

    public int insertAfter(int slot) {
        int newslot = searchAfter(slot, EMPTY);
        if (newslot >= 0) {
            setFlag(newslot, USED);
        }
        return newslot;
    }

    ...
    public int getInt(int slot, String fldname) {
        int fldpos = offset(slot) + recordValueLayout.offset(fldname);
        return tx.getInt(blockId, fldpos);
    }

    ...
    // Store an integer at the specified field of the specified slot.
    // Here slot is the recordIdx within the Block.
    public void setInt(int slot, String fldname, int val) {
        int fldpos = offset(slot) + recordValueLayout.offset(fldname);
        tx.setInt(blockId, fldpos, val);
    }

    ...
    public void delete(int slot) {
        setFlag(slot, EMPTY);
    }

    ...
}
```

3. Transaction

Ideally, the transaction should have `commit()`, `pin()`, etc. But to make things simple, we are ignoring it in the current version. Transaction internally uses Page object to read and write to/from the file manager.

```
public class Transaction {
    private static int nextTxNum = 0;
    private FileMgr fm;
    public Transaction(FileMgr fm) {this.fm = fm; txnum = nextTxNumber(); }

    // Monotonically increasing Txn Number
    private static synchronized int nextTxNumber() {
        nextTxNum++;
        return nextTxNum;
    }

    ...

    public synchronized int getInt(BlockId blk, int offset) {
        Page contents = new Page(fm.blockSize());
        fm.read(blk, contents);
        return contents.getInt(offset);
    }

    ...

    public synchronized void setInt(BlockId blk, int offset, int val) {
        Page contents = new Page(fm.blockSize());
        fm.read(blk, contents);

        contents.setInt(offset, val);
        fm.write(blk, contents);
    }
}
```

4. Page

Page is the in-memory representation of a Block saved in the Disk. We use it to read and update block data.

```
public class Page {
    public static Charset CHARSET = StandardCharsets.US_ASCII;
    private ByteBuffer bb;

    public Page(int blocksize) { bb = ByteBuffer.allocateDirect(blocksize); }
    ...
    public int getInt(int offset) { return bb.getInt(offset); }

    public void setInt(int offset, int n) { bb.putInt(offset, n); }
    ...
    public byte[] getBytes(int offset) {
        bb.position(offset);
        int length = bb.getInt();
        byte[] b = new byte[length];
        bb.get(b);
        return b;
    }

    public String getString(int offset) {
        byte[] b = getBytes(offset);
        return new String(b, CHARSET);
    }
    ...
    private ByteBuffer contents() {
        bb.position(0);
        return bb;
    }
}
```

5. File Manager

File Manager is the class responsible for all the File IO.

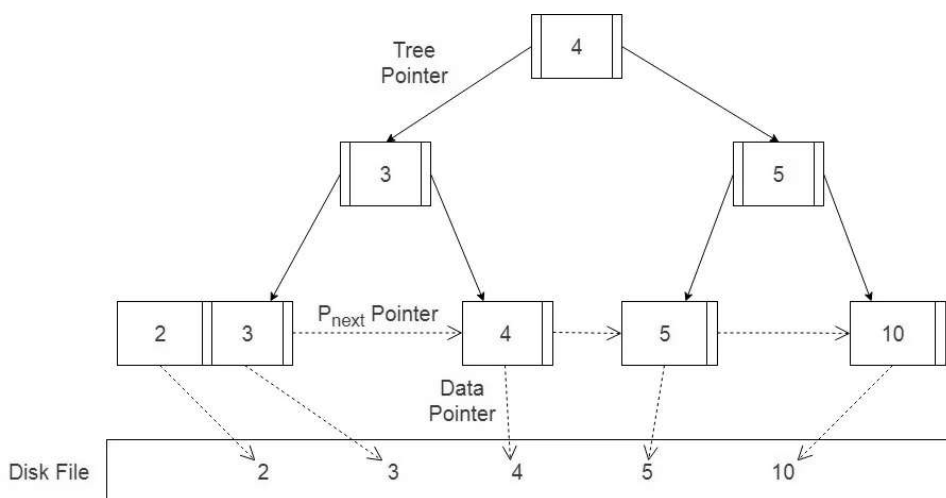
```
public class FileMgr {
    private final File dbDirectory;
    private final int blockSize;
    private final boolean isNew;

    public synchronized void read(BlockId blk, Page p) {
        try {
            RandomAccessFile f = getRandomAccessFile(blk.getFileName());
            f.seek((long) blk.getBlockNumber() * blockSize);
            f.getChannel().read(p.contents());
            f.close();
        } catch (IOException e) {
            throw new RuntimeException("cannot read block " + blk);
        }
    }

    public synchronized void write(BlockId blk, Page p) {
        try {
            RandomAccessFile f = getRandomAccessFile(blk.getFileName());
            f.seek((long) blk.getBlockNumber() * blockSize);
            f.getChannel().write(p.contents());
            f.close();
        } catch (IOException e) {
            throw new RuntimeException("cannot write block" + blk);
        }
    }

    public int blockCount(String filename) {
        try {
            RandomAccessFile f = getRandomAccessFile(filename);
            int result = (int) (f.length() / blockSize);
            f.close();
            return result;
        } catch (IOException e) {
            throw new RuntimeException("cannot access " + filename);
        }
    }
}
```

Index Module



The index implemented on top of the **Heap File Organization** helps in improved performance, if the query uses an indexed column.

In our project, the B+Tree index class implements this interface.

```
public interface RWIndexScan {

    public void insert(D_Constant key, RecordKey value);

    public void delete(D_Constant key, RecordKey value);

    // Iterator
    public void seek(D_Constant key);
    public boolean hasNext();
    public RecordKey next();
    public void close();
}
```

For concrete implementation, we could use any of the open-source B+Tree libraries.

```
// B+ tree index using https://github.com/davidmoten/bplustree
public class BPlusTreeIndex implements RWIndexScan {

    BPlusTree<D_Constant, RecordKey> tree;
    Iterator<RecordKey> iterator;

    public AdvancedBPlusTreeIndex() {
        tree = BPlusTree.file().directory("tinydb")
            .maxLeafKeys(32)
            .maxNonLeafKeys(8)
            .segmentSizeMB(1)
            .keySerializer(new ConstantSerializer())
            .valueSerializer(new RecordKeySerializer())
            .naturalOrder();
    }

    @Override
    public void insert(D_Constant key, RecordKey value) {
        tree.insert(key, value);
    }

    @Override
    public void seek(D_Constant key) {
        iterator = tree.find(key).iterator();
    }

    @Override
    public boolean hasNext() {
        return iterator.hasNext();
    }

    @Override
    public RecordKey next() {
        return iterator.next();
    }

    @SneakyThrows
    @Override
    public void close() {
        tree.close();
    }
}
```